

ECS401: Unit 2 Lab Progress Test

FEEDBACK: Answers and Explanations

You may have got some of the answers wrong, but if so don't worry. Learn from your mistakes. You are capable of sorting it out but you need to put some effort in now. It just takes a bit of work on your part and some help if needed. Once you have understood the concepts properly you just then need a bit of care thinking things through when you are answering questions.

If you take it seriously, this sheet will help improve your understanding and the lab tutors can give you more help if you need it.

1. What is the value in the variable `answer` after the following fragment of code has been executed?

```
int answer = 55;
```

CORRECT ANSWER:

55

EXPLANATION:

The code declares a new variable of type `int` and immediately initialises it with the value 55. Put another way what that means is a new 'box' for storing things is created. It is made of appropriate size and shape to hold an integer (a whole number) and the number 55 is stored in it.

For questions 2 and 3 first consider the following fragment of code and complete the dry run table:

```
int a;  
int b;  
a = 2; // L1  
b = 3; // L2  
b = a; // L3
```

Line	a	b
L1	2	
L2		3
L3		2

2. What is the value in the variable `a` after the following fragment of code has been executed?

CORRECT ANSWER:

2

EXPLANATION:

The first assignment (`a = 2;`) places the number 2 into variable `a`. None of the other

commands put anything into a (b is on the left hand side of the assignment so it is the thing that values are put into). That means at the end after each instruction has been executed in turn a still holds the value 2.

If you thought the answer was 3 you may be getting mixed up about the = sign. It doesn't mean two things are equal (so don't think of it as the word "equals"), it is an instruction to store a new value. It's better to read it as "gets the value". It make a copy of the result of a calculation and store that value in the variable on the left hand side. Read (b = a;) as "b gets a copy of the value that is currently stored in a".

You may also have just got mixed up about which way round the data moves. The first part of an assignment is always a destination to put something so for example

b = 1; makes sense but 1 = b; is rubbish because 1 is a value not a variable so not a place where something can be stored.

3. What is the value in the variable b after the following fragment of code has been executed?

CORRECT ANSWER:

2

EXPLANATION:

The instructions are carried out in the order they appear. That means the last value put into variable b is whatever is held in a as the last instruction is executed. As we saw in question 2 a holds the value 2 at this point so a copy of the value 2 is made and put into b as well.

If you thought that b was still 3 you may have not been doing the commands in order. b did have the value 3 earlier but by the end that value has been lost as a new value was inserted by the last instruction.

4. What is the value in the variable b after the following fragment of code has been executed?

CORRECT ANSWER:

It does not have a value.

EXPLANATION:

All that this line does is declare the variable not put a value into it. In other words by executing this instruction you have just created a 'box' to store things in, but it is

still empty as you haven't put anything in to it yet. It needs to be initialised by an assignment statement. If you write a program that uses a variable that has only been declared not initialised then the Java compiler will just complain – you do not have a valid program to run. In some situations Java does initialise variables by default but not local primitive variables as here. For example, global variables are initialised but you should not use them anyway and certainly not uninitialised. (That is why just declaring a variable in JHUB outside a method gives it a default value but you should not be doing that in programs.)

5. What is the value in the variable d after the following fragment of code has been executed?

```
int d = 2 + 3 * 4;
```

CORRECT ANSWER:

14

EXPLANATION:

This instruction does a calculation and stores the result into variable d. However, calculations are done in a special order, not just the order as read. It follows the school BODMAS rule. Things in brackets are calculated first, then multiplication and division and finally addition and subtraction. In our case 3 is multiplied by 4 first to get 12 and then 2 is added.

If you thought the answer was 20 then you just did the calculations as you came to them – be careful and remember that multiplications are done before additions. If you wrote any other answer then you need to be more careful doing arithmetic and if it wasn't just a silly slip get some maths support urgently.

6. What is printed to the screen after the following has been executed?

```
String e = "a";  
System.out.println("b" + e + "d");
```

CORRECT ANSWER:

bad

EXPLANATION:

The variable called e is given the value "a". That is a box we happen to be calling e, has a single character "a" stored inside it. Before anything is printed the println method turns the things it is given in the brackets into a string. In this case it is given a "b" followed by whatever is inside e followed by a "d" to stick together in that

order. That means it glues together a "b" an "a" and a "d" to get the single string "bad" which is printed to the screen. The cursor then moves onto the next line as though a return had been hit.

If you thought the answer was bed you are confusing the name of a variable (e) with the thing that is stored in it. Calculations with variables involve getting the value out and doing something with that not doing something with the name.

If you thought the answer was "bad" with quotes round then you need to realise that the quotes of a string are just used in a program as start and end markers so the computer can tell what is and isn't a string. They are not part of what is stored or printed. If you want to store or print a quote character you have to give a specific command to do that.

For questions 7 and 8, first consider the following fragment of code and complete the dry run table:

```
int f = 1; // L1
int g = 2; // L2
int h = 3; // L3
h = f      // L4
f = g;     // L5
g = h;     // L6
```

Line	f	g	h
L1	1		
L2		2	
L3			3
L4			1
L5	2		
L6		1	

7. What is the value in variable f after the above has been executed?

CORRECT ANSWER:

2

8. What is the value in variable g after the above has been executed?

CORRECT ANSWER:

1

EXPLANATION (7 & 8):

This is standard code to swap the value of two variables over (in this case exchange what is in f and what is in g). h is just a place holder to temporarily hold f's value so it is not lost when the 2 from g is stored there as assignment destroys whatever was previously destroyed.

If you thought that both variables end up with the same value you are may still be reading = as “equals” instead of as a command to copy values from place to place.

You might want to make sure you understand why the following doesn’t swap the values over. It does just leave them both with the same value (can you work out which value)?

```
int f = 1;
int g = 2;
g = f;
f = g;
```

Step through it line by line to see what happens.

9. Write a fragment of code that declares 4 variables, initialising them to 10, 20, 30 and 40 and then stores their average in a fifth variable.

CORRECT ANSWER:

Here is one answer – the variable names could be different.

```
int v1 = 10;
int v2 = 20;
int v3 = 30;
int v4 = 40;
int average = (v1 + v2 + v3 + v4) / 4;
```

This code always gives rounded down answers. For an accurate answer you need to use a different type for the numbers (not int but double).

Terminology

10. A **procedure call** is where a previously defined piece of code (a method) is executed by giving its name. Procedures *do not* return values. Highlight/underline all the places where procedure calls appear in the following code.

```
public static void main (String[] param)
{
    helloMessage ();
    return;
} // END main
```

```

public static void helloMessage ()
{
    System.out.println("Hello World!");
    return;
} // END helloMessage

```

Note that the place where the method code is actually given is not the *call*. That is the declaration of the method. The call is the place where you say you want to execute the code.

There is one call to a procedure that the programmer has defined and another that are pre-defined in Java.

11. A **function call** is where a previously defined piece of code (a method) is executed by giving its name. Functions *do* return values. Highlight/underline all the places where function calls appear in the following code. public static void main (String[] param)

```

{
    String message = helloMessage();
    System.out.println(message);
    return;
} // END main

```

```

public static void helloMessage ()
{
    String m = inputString("Enter a message");
    return m;
} // END helloMessage

```

Here there are two function calls. In each case the value returned is caught and stored in a variable for use in the next line.

Note that in the declaration of a method, a procedure has the word void in its header line before the method name (see main above). In the declaration of a function that is replaced by the type being returned (see the type String given before the name helloMessage above).